BENCHMARK NOTES · CONE-BEAM CT POSE SEARCH

OpenCL vs C++ Backend Benchmark

Ten repeated runs of the full search and resample pipeline for both backends, on both datasets. All numbers below are measured wall time from repeated runs, not single samples.

---

01 Result

Which backend wins depends on dataset size, and it flips. Ten runs each, taken back to back with nothing else competing for the GPU.

OPENCL		CPP
2.41s	cpp 15% faster	**2.10s**
mean of 10 runs		mean of 10 runs

128×128, 500 projections

OPENCL		CPP
11.21s	opencl 46% faster	**16.36s**
mean of 10 runs		mean of 10 runs

800×500, 1000 projections

DATASET	BACKEND	RUNS (S)										MEAN	MIN	MAX
128px	opencl	3.23	2.21	2.25	2.72	2.47	2.37	2.47	2.18	2.24	1.98	2.41s	1.98s	3.23s
128px	**cpp**	**1.89**	**2.86**	**3.04**	**2.46**	**1.84**	**1.82**	**1.72**	**1.92**	**1.46**	**1.95**	**2.10s**	**1.46s**	**3.04s**
512px	**opencl**	**11.16**	**10.49**	**11.09**	**11.97**	**12.64**	**10.67**	**10.35**	**10.20**	**11.37**	**12.17**	**11.21s**	**10.20s**	**12.64s**
512px	cpp	14.88	16.39	16.59	18.17	14.42	15.43	18.09	16.52	17.17	16.00	16.36s	14.42s	18.17s

ⓘ `projection-center pipeline --backend X`, timed with `time.perf_counter()` around each subprocess call, 10 repeats per backend per dataset.

02 What was ruled out as the cause

Resample kernel code

The C++ version and the Python version run the exact same kernel. Same math, same steps, nothing different there.

GPU buffer caching

Both backends set up their GPU memory once at the start and reuse it for every batch, instead of setting it up fresh each time. Both work the same way here, so this is not where the difference comes from.

03 What batching bigger transfers changes

Resample only, no search, 128px, 10 runs each, comparing the default `--batch-size` 16 against one single transfer covering the whole dataset.

BACKEND	BATCH=16	BATCH=FULL	CHANGE
opencl	**3.07s**	**2.30s**	**25% faster**
cpp	2.31s	2.13s	8% faster

Doing fewer, bigger transfers speeds up both backends, but it helps opencl about three times more than it helps cpp. This closes part of the gap at the small dataset size, but not all of it.

04 Where the two backends actually differ

`strace -c`, aggregated over 10 runs each, resample only, 128px.

BACKEND	FUTEX TIME (10 RUNS)	SHARE OF TRACED TIME	CALLS
cpp	4.70s	52.6%	1539
opencl	40.10s	88.3%	2032

A futex is just a thread waiting its turn, not actual work being done. The opencl backend uses more internal threads behind the scenes than the cpp backend does, which shows up as more waiting. cpp also has to start up fresh as a new program on every run, while opencl stays running and ready.

05 Open question

Which backend wins really does depend on dataset size, and that part is confirmed. But there is still no clear single reason why it flips. The kernel code and the GPU memory setup are the same for both, and the threading difference found above does not fully explain it either. Figuring out the real cause would need a more focused test that measures raw processing speed at both sizes, not just total time.